Qualcomm Technologies, Inc.

AudioReach Bluetooth Encoder

User Guide

80-VN500-21 Rev. AA

December 23, 2020

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 09:35:08 GMT lijiasheng@longcheer.com

For additional information or to submit technical questions, go to: https://createpoint.qti.qualcomm.com

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

Restricted Distribution: Not to be distributed to anyone who is not an employee of either Qualcomm Technologies, Inc. or its affiliated companies without the express approval of Qualcomm Configuration Management. Distribution to anyone who is not an employee of either Qualcomm Incorporated or its affiliated companies is subject to applicable confidentiality agreements.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies, Inc.

All Qualcomm products mentioned herein are products of Qualcomm Technologies, Inc. and/or its subsidiaries.

Qualcomm is a trademark of Qualcomm Incorporated, registered in the United States and other countries. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer (“export”) laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
U.S.A.

© 2020 Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision history

Revision	Date	Description
AA	December 2020	Initial release

Qualcomm
Confidential - May Contain Trade Secrets
2022-11-05 09:35:08 GMT
nijiasheng@longcheer.com

Contents

1 Introduction 5

1.1 Purpose 5
1.2 Conventions 5
1.3 Technical assistance 5

2 Bluetooth encoder in the Elite framework 6

3 Bluetooth encoder in AudioReach 8

3.1 Control ports 8
3.1.1 Drift information from Rx HW-EP module to RAT-EP 8
3.1.2 Encoder feedback information from COP Depacketizer to Encoder 8
3.2 Data flow 9
3.3 Prerequisite configurations for Bluetooth use cases 9
3.4 ACDB graph 10
3.5 AP call flow 11

4 Debugging 12

4.1 Enable logging 12
4.2 Logcat analysis 12
4.2.1 GKV analysis 12
4.2.2 AHAL analysis 17
4.2.3 QAL analysis 17
4.2.4 AGM analysis 18
4.3 QXDM analysis 19
4.4 Common issues 22

5 Customize the encoder 23

5.1 Add entry in the .json file 23
5.2 Add API header file path in the .scons file 23
5.3 Define control port 24
5.4 Implement control port 25
5.4.1 Register CAPI_INTERFACE_EXTENSIONS 25
5.4.2 Port configuration 26
5.4.3 Receiving data from sender 33
5.4.4 Sending data to the receiver 35
5.5 Metadata 36
5.6 AP side customization 37
5.7 ACDB design 38

Qualcomm Confidential – May Contain Trade Secrets 2022-11-05 09:35:08 GMT hucheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

A References 39
A.1 Related documents 39
A.2 Acronyms and terms 39

Qualcomm
Confidential - May Contain Trade Secrets
2022-11-05 09:35:08 GMT
nijiasheng@longcheer.com

1 Introduction

1.1 Purpose

This document describes Bluetooth encoder implementation and customization in the AudioReach™ framework.

1.2 Conventions

Function declarations, function names, type declarations, attributes, and code samples appear in a different font, for example, `cp armcc armcpp`.

Code variables appear in angle brackets, for example, `<number>`.

Commands to be entered appear in a different font, for example, **`copy a:*.* b:`**.

Button and key names appear in bold font, for example, click **Save** or press **Enter**.

Shading indicates content that has been added or changed in this revision of the document.

1.3 Technical assistance

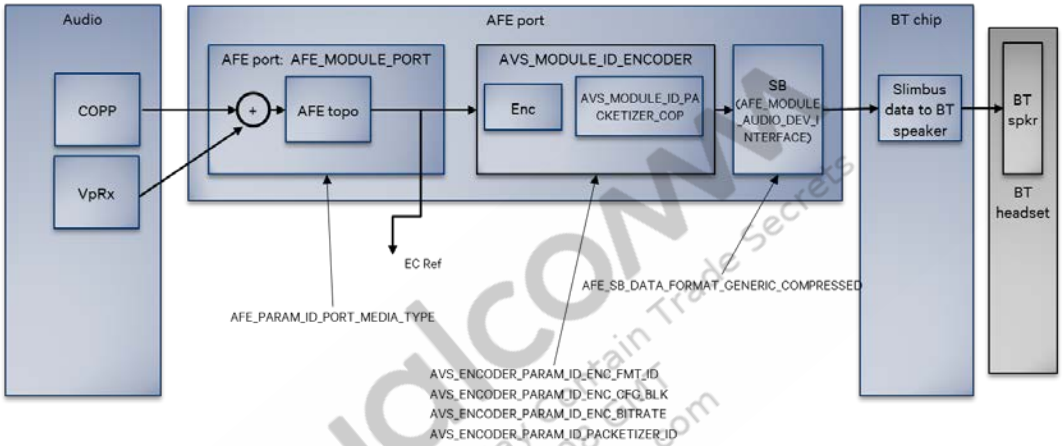
For assistance or clarification on information in this document, submit a case to Qualcomm Technologies, Inc. (QTI) at https://createpoint.qti.qualcomm.com/.

If you do not have access to the CDMATech Support website, register for access or send email to support.cdmatech@qti.qualcomm.com.

Qualcomm Confidential - May Contain Trade Secrets
2022-11-05 09:35:58 GMT
nijian.ong@longcheer.com

2 Bluetooth encoder in the Elite framework

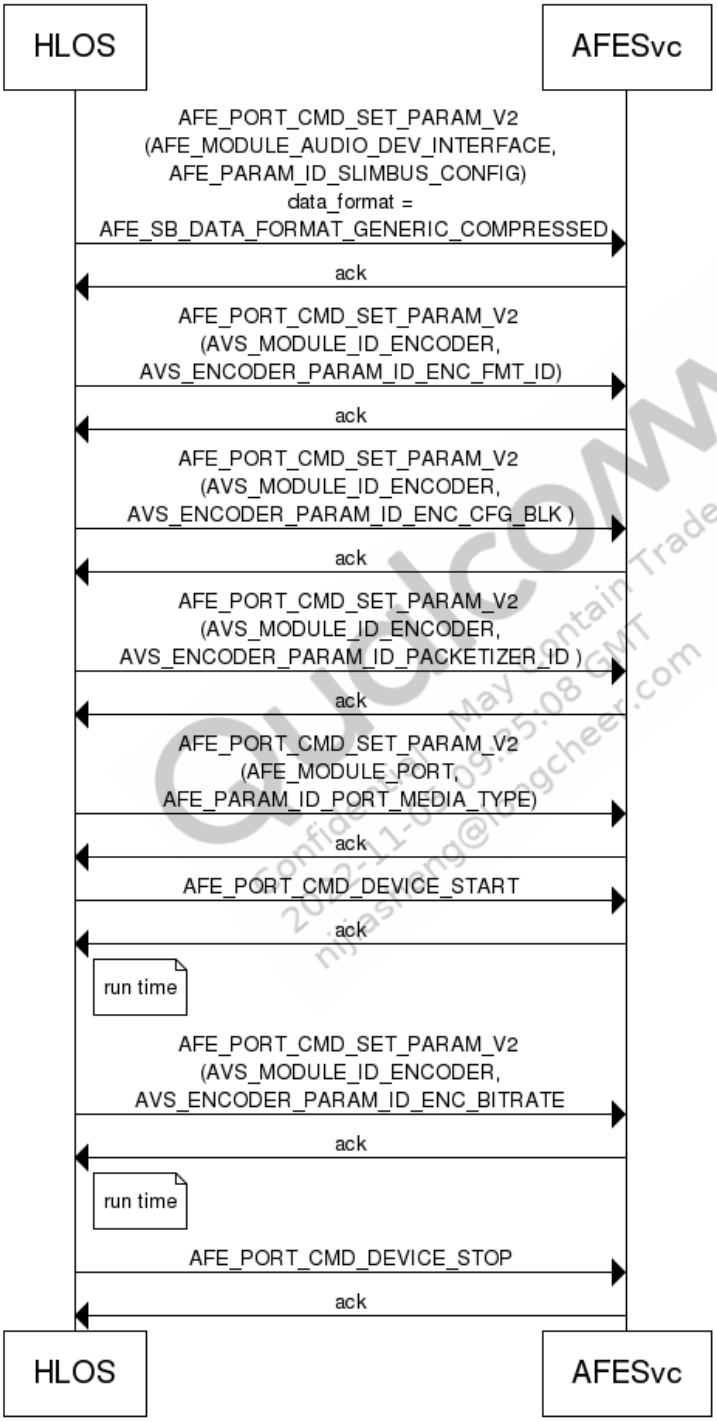
The following diagram illustrates split A2DP in the Elite framework.



The encoder is present in the AFE topology, and the encoder module can be customized using the AVS_MODULE_ID_ENCODER block. The HLOS can configure the module with AFE_PORT_CMD_SET_PARAMS_V2.

Qualcomm Confidential – May Contain Trade Secrets 2022-11-07 07:06:05 nijiasheng@qualcomm.com

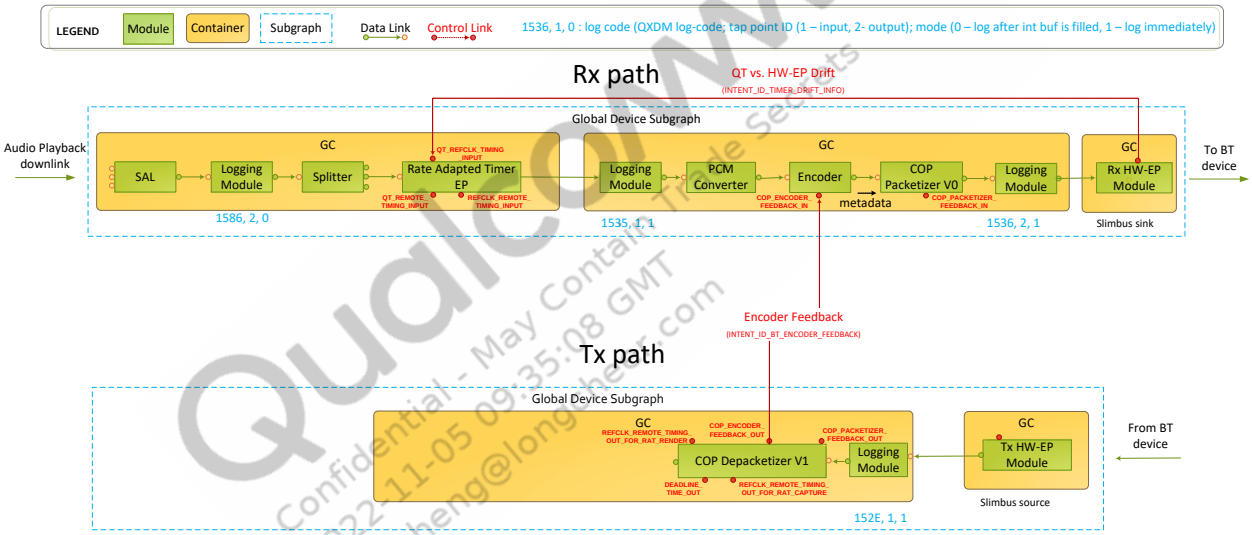
The following diagram shows the access point (AP) configuration sequences for split A2DP in the Elite framework.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3 Bluetooth encoder in AudioReach

The following diagram is the subgraph for split A2DP in AudioReach. The main modules are Rate Adapted Timer endpoint (RAT-EP) and Encoder in the Rx path and compressed over PCM (COP) Depacketizer in the Tx path. This diagram does not show other subgraphs in the general playback path.



3.1 Control ports

3.1.1 Drift information from Rx HW-EP module to RAT-EP

As the HW-EP clock source (by default, SLIMbus) is different from Qtimer, drifts could occur. RAT-EP uses the information to align the timeline for data transportation.

3.1.2 Encoder feedback information from COP Depacketizer to Encoder

Some encoders may need Tx feedback information for adaptive rate encoding.

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 09:35:08 GMT nijiasheng@longtheer.com

3.2 Data flow

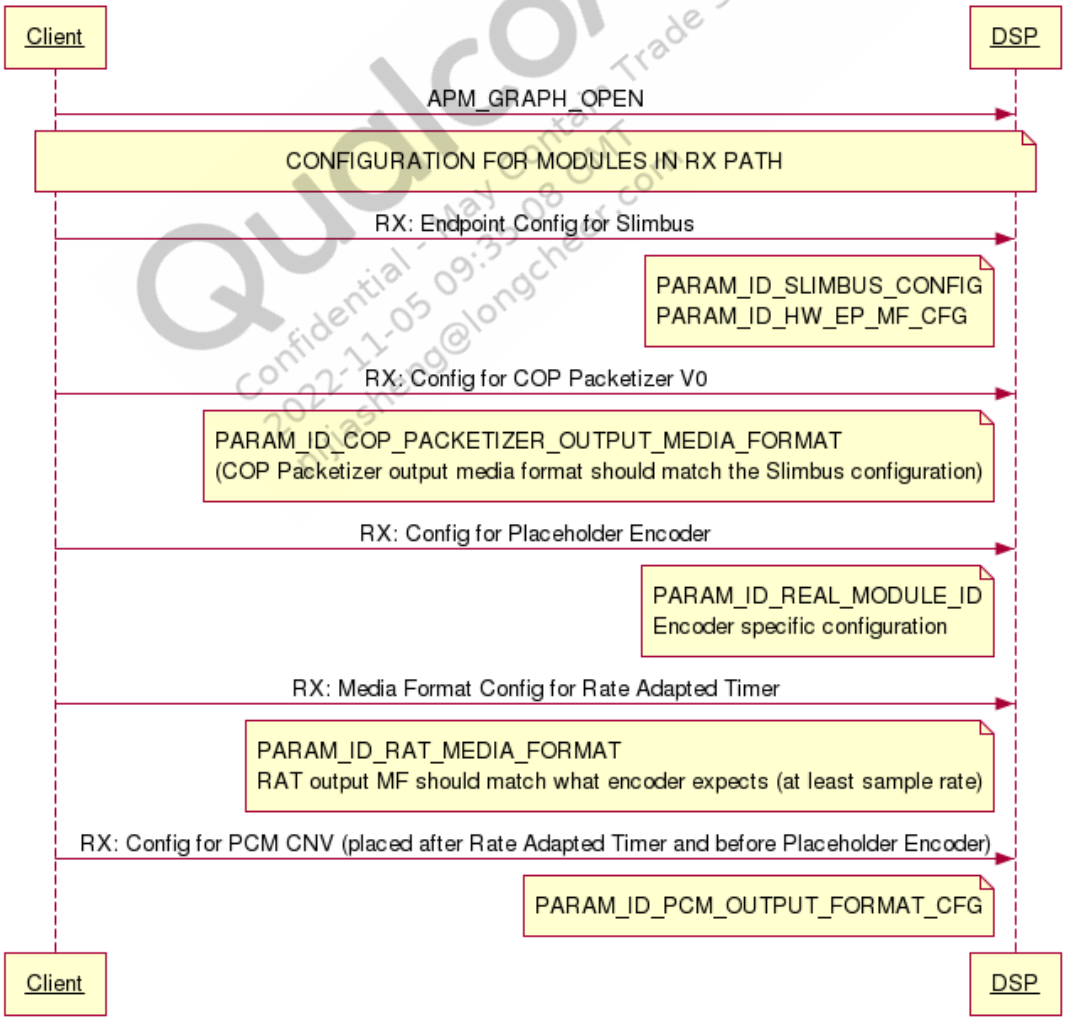
In the Rx path, the COP packetizer media format must be configured to be aligned with the Rx HW-EP module's media format. On SM8450, the default SLIMbus sample rate is set at 96 KB/s and the channel as mono. The RAT-EP media format needs to be aligned with the encoder module's media format, which can be stereo/mono or multichannel.

By default, the APTX and LDAC encoder are supported.

3.3 Prerequisite configurations for Bluetooth use cases

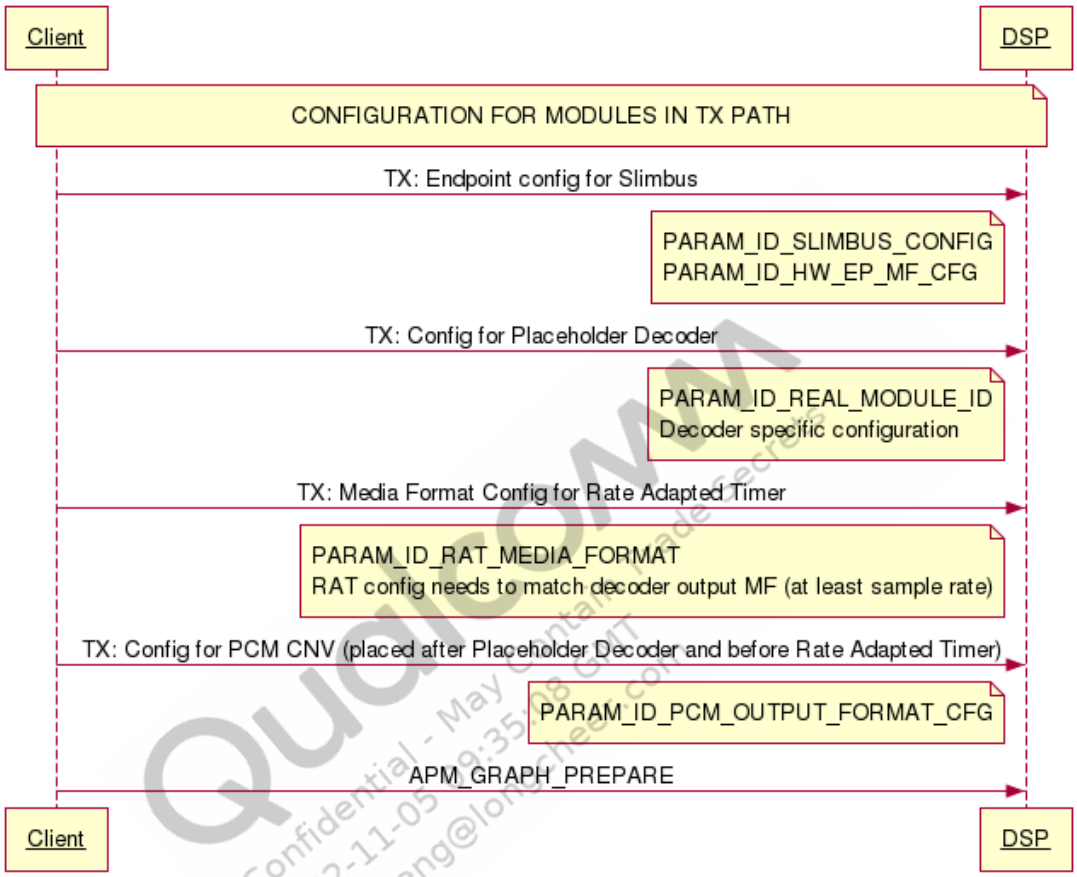
Before starting Bluetooth use cases, the GKV from the AP side and some module-specific parameters must be configured. After this is done, the split A2DP subgraph can be opened and started along with the playback subgraphs.

The following diagram shows the module parameters that must be configured in the Rx path.



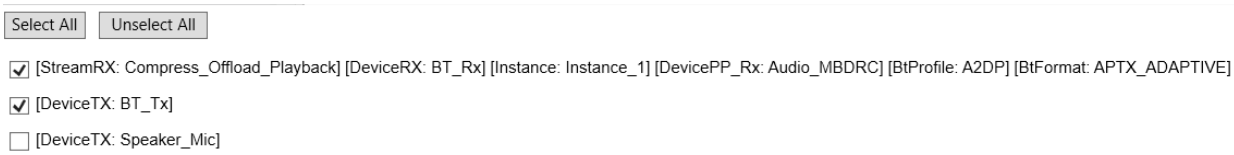
MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The following diagram shows the module parameters that must be configured in the Tx path.



3.4 ACDB graph

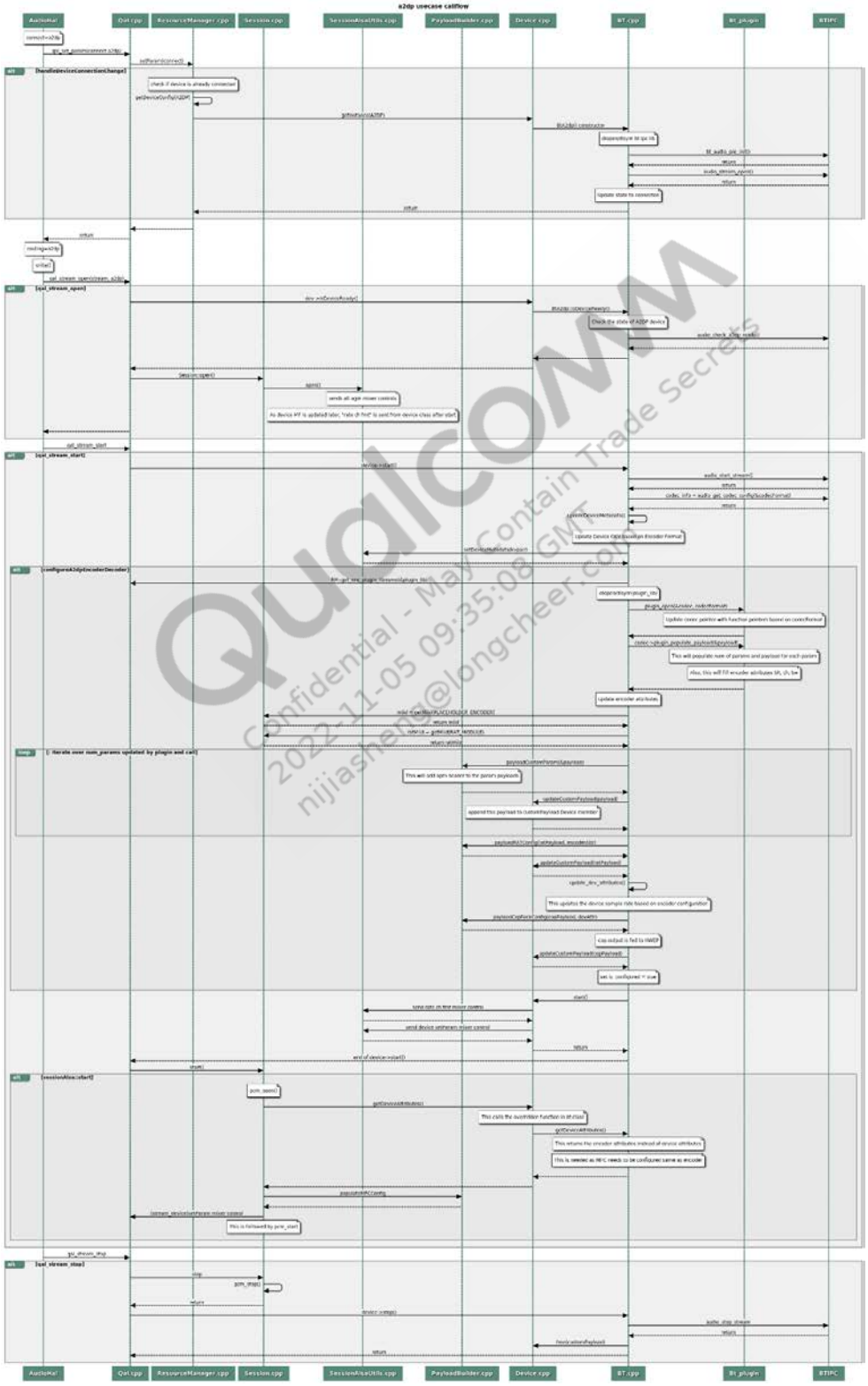
The following two use cases should be selected.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.5 AP call flow

The following diagram shows the A2DP playback sequence. The AP call flow is similar to normal audio playback but also includes Bluetooth-specific module configuration.



Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 09:35:08 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4 Debugging

4.1 Enable logging

Set the following to enable logging:

- QXDM
  - 0x1586
  - 0x1531
  - 0x1532
  - 0x19B0
  - QDSP6 messages
- For Logcat, run the following command:

```
adb logcat -v threadtime
```

4.2 Logcat analysis

The following definitions can be found in file mm-audio/ar-acdb/acdbdata/inc/kvh2xml.h

4.2.1 GKV analysis

To set up the Bluetooth encoder graph, GKVs must be sent from the AP side because the encoder may need Tx path data as feedback for rate information updates.

Both Rx and Tx use cases are needed – 2 use cases (Rx/Tx) and 4 metadata (string "GKV").

For Rx, there are 3 metadata. For Tx, there is 1 metadata.

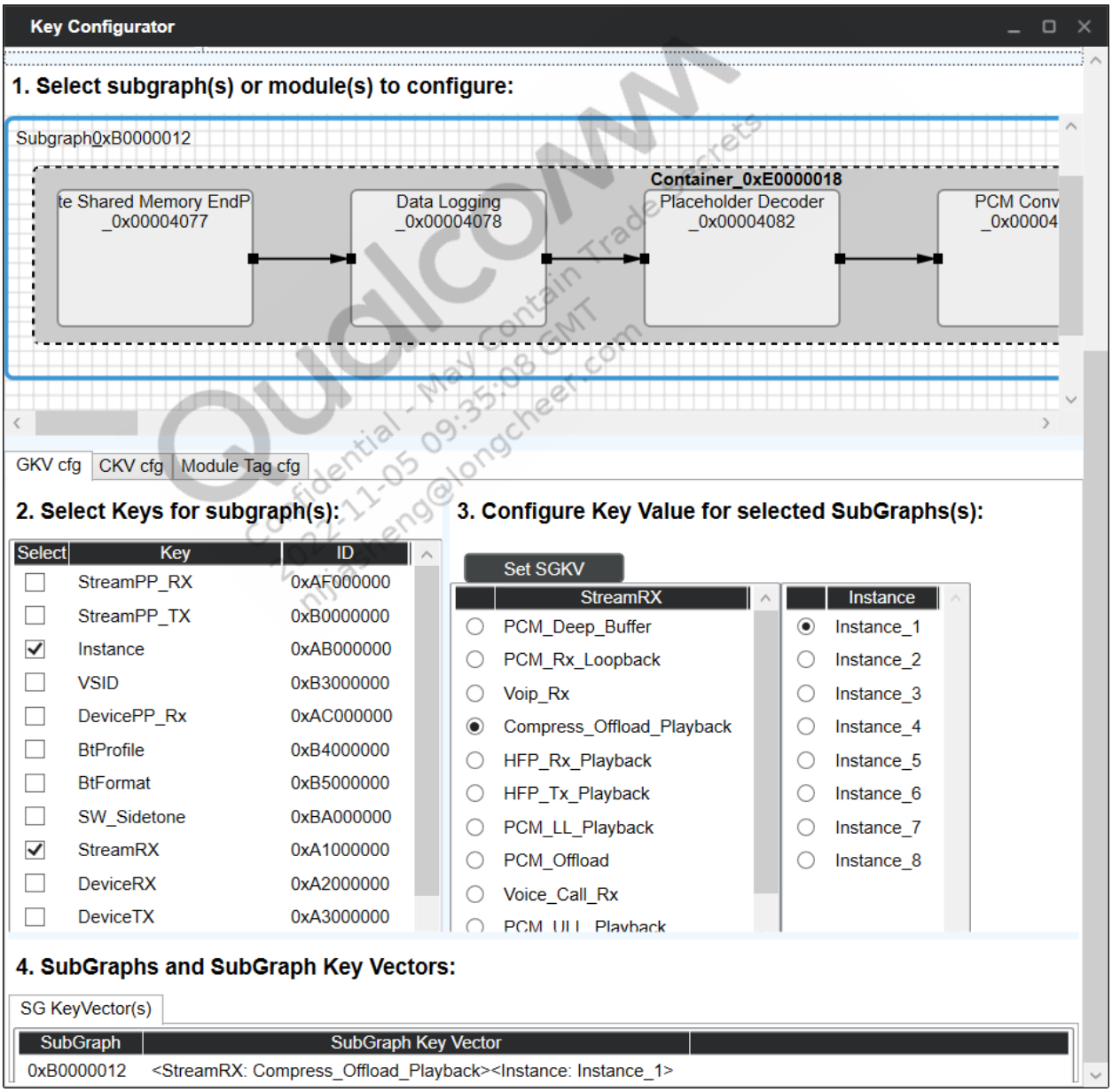
```
09-12 04:02:48.849   862  1491 D AGM: metadata: metadata_print: 78 GKV
size:2
09-12 04:02:48.850   862  1491 D AGM: metadata: metadata_print: 78 GKV
size:1
09-12 04:02:49.376   862  1491 D AGM: metadata: metadata_print: 78 GKV
size:3
09-12 04:02:49.396   862  1487 D AGM: metadata: metadata_print: 78 GKV
size:1
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 09:35:08 GMT jijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

For the BT_RX use case, the STREAM_RX GKV must be configured as shown.

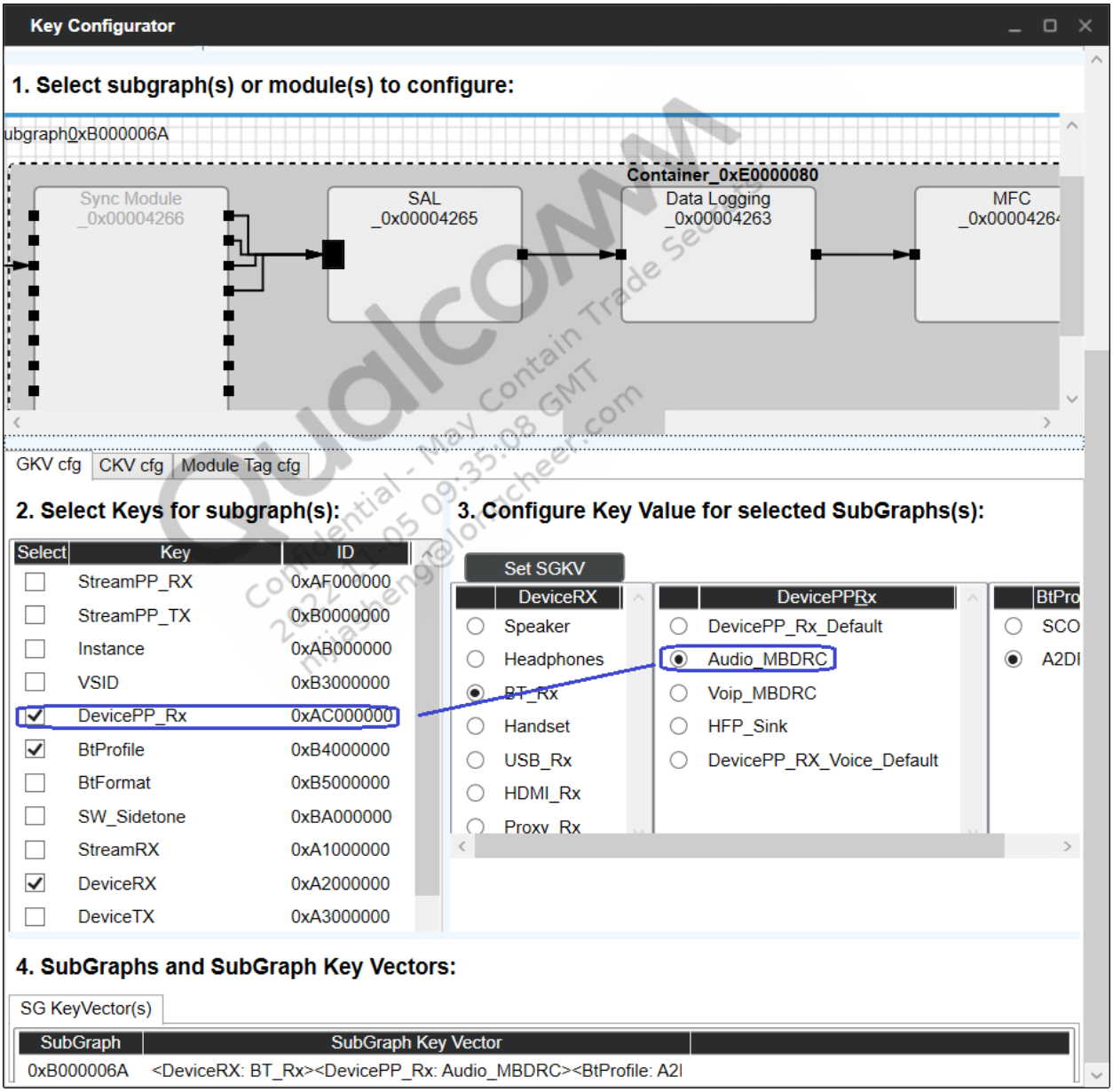
```
************************Metadata************************
09-12 04:02:48.849   862  1491 D AGM: metadata: metadata_print: 78 GKV
size:2
09-12 04:02:48.849   862  1491 D AGM: metadata: metadata_print: 81
key:0xa1000000, value:0xa100000a
09-12 04:02:48.849   862  1491 D AGM: metadata: metadata_print: 81
key:0xab000000, value:0x1
```



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

For the BT_RX use case, the DEVICE_PP_RX GKV must be configured as shown.

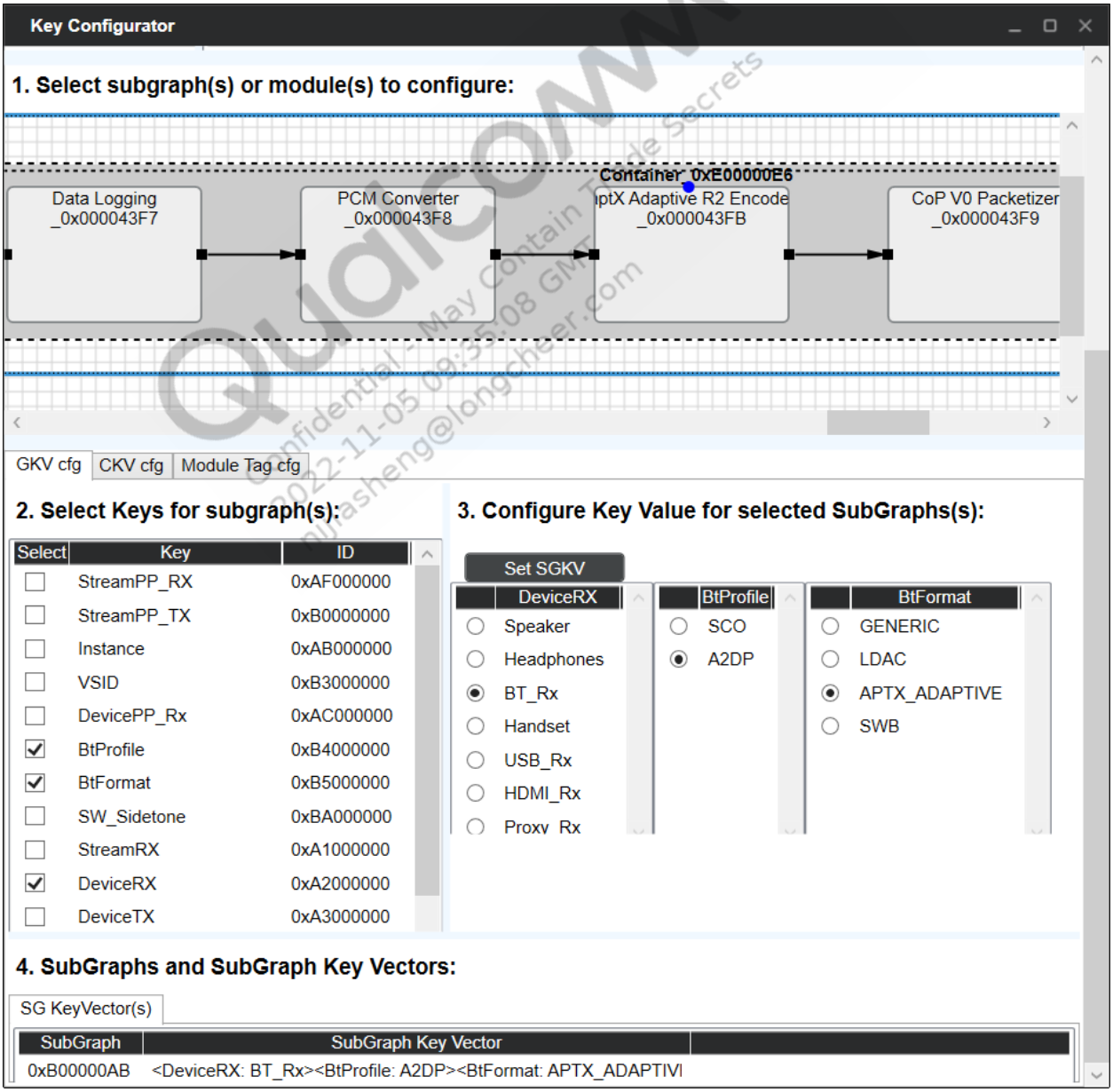
```
************************Metadata************************
09-12 04:02:48.850   862  1491 D AGM: metadata: metadata_print: 78 GKV
size:1
09-12 04:02:48.850   862  1491 D AGM: metadata: metadata_print: 81
key:0xac000000, value:0xac000002
```



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

For the BT_RX use case, the DEVICE_RX GKV must be configured as shown.

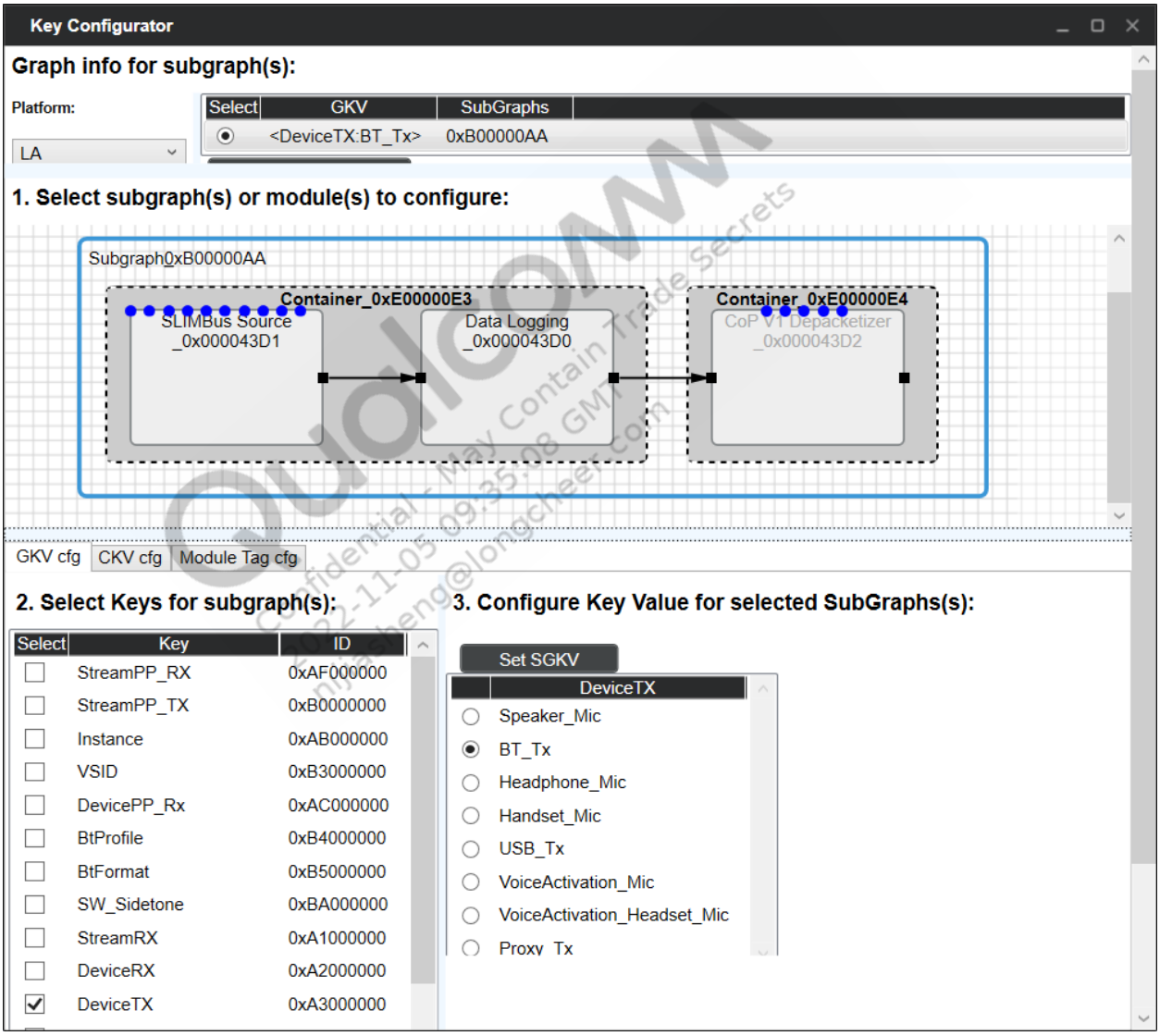
```
************************Metadata************************
09-12 04:02:49.376   862  1491 D AGM: metadata: metadata_print: 78 GKV
size:3
09-12 04:02:49.376   862  1491 D AGM: metadata: metadata_print: 81
key:0xa2000000, value:0xa2000003
09-12 04:02:49.376   862  1491 D AGM: metadata: metadata_print: 81
key:0xb4000000, value:0xb4000002
09-12 04:02:49.376   862  1491 D AGM: metadata: metadata_print: 81
key:0xb5000000, value:0xb5000003
```



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

For the BT_TX use case, the Device_TX GKV must be configured as shown.

```
************************Metadata************************
09-12 04:02:49.396   862  1487 D AGM: metadata: metadata_print: 78 GKV
size:1
09-12 04:02:49.396   862  1487 D AGM: metadata: metadata_print: 81
key:0xa3000000, value:0xa3000002
```



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.2.2 AHAL analysis

```
141:09-12 04:02:48.583   802  5752 E ahal_AudioStream: StreamPrimary:
handle: 149 channel_mask: 3
142:09-12 04:02:48.583   802  5752 E ahal_AudioStream: StreamOutPrimary:
enter: handle (95) format(0x1b000000) sample_rate(96000) channel_mask(0x3)
devices(0x80) flags(0x31)          address(00:02:5B:00:FF:01)
197:09-12 04:02:48.717   802  2131 D ahal_AudioStream:
astream_out_set_parameters: enter: usecase(3: compress-offload-playback)
kvpairs: a2dp_sink_address=00:02:5B:00:FF:01;routing=128
```

4.2.3 QAL analysis

```
272:09-12 04:02:48.847   802  6505 E QAL: Stream: create:156 stream
0xef782920K created
//Front end
274:09-12 04:02:48.847   802  6505 I QAL: ResourceManager:
allocateFrontEndIds:3118 allocateFrontEndIds: front end 105
//Back end
275:09-12 04:02:48.848   802  6505 E QAL: ResourceManager:
getBackEndNames:3409 getBackEndNames (RX): SLIM-DEV1-RX-7
276:09-12 04:02:48.848   802  6505 E QAL: Stream: getAssociatedDevices:255
no. of devices 1
277:09-12 04:02:48.848   802  6505 E QAL: SessionAlsaUtils:
getFeMixerControl:265 mixer control COMPRESS105 control
278:09-12 04:02:48.848   802  6505 E QAL: SessionAlsaUtils:
getFeMixerControl:265 mixer control COMPRESS105 metadata
279:09-12 04:02:48.848   802  6505 E QAL: SessionAlsaUtils:
getFeMixerControl:265 mixer control COMPRESS105 connect
299:09-12 04:02:48.850   802  6505 I QAL: PayloadBuilder:
populateDevicePPCkv:1579 SpeakerProt Status[0], RAS Status[0] device id[8]
316:09-12 04:02:48.852   802  6505 E QAL: StreamCompress: open:153 device 8
name QAL_DEVICE_OUT_BLUETOOTH_A2DP, going to open
328:09-12 04:02:48.852   802  6505 E QAL: StreamCompress: start:316 device
8 name QAL_DEVICE_OUT_BLUETOOTH_A2DP, going to start
//Start Rx playback path
493:09-12 04:02:49.371   802  6505 E QAL: Bluetooth: startPlayback:935 BT
controller start return = 0
540:09-12 04:02:49.375   802  6505 I QAL: Bluetooth:
updateDeviceMetadata:83 Setting BT_FORMAT = APTX_ADAPTIVE
//Configure the BT encoder
570:09-12 04:02:49.377   802  6505 I QAL: Bluetooth:
configureA2dpEncoderDecoder:245 configureA2dpEncoderDecoder: choose BT
codec format 620756992
598:09-12 04:02:49.395   802  6505 I QAL: SessionAlsaUtils:
setDeviceMediaConfig:669 SLIM-DEV1-RX-7 rate ch fmt data_fmt 96000 1 2 7
616:09-12 04:02:49.397   802  6505 I QAL: SessionAlsaUtils:
setDeviceMediaConfig:669 SLIM-DEV1-TX-7 rate ch fmt data_fmt 8000 1 2 7
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
//Start TX loopback
657:09-12 04:02:49.470   802  6505 I QAL: Bluetooth: startAbr:623 Feedback
Device started successfully
```

4.2.4 AGM analysis

```
//1. tx graph : 0x47534c
627:09-12 04:02:49.434   862  4476 D AGM: graph: graph_open: 425 entry
//configure TX HW EP
//tkv: c0000005 = device_hw_ep_tx
628:09-12 04:02:49.435   862  4476 D AGM: graph: graph_open: 537 miid 43d1
mid 700102a tag c0000005
//graph_open->graph_prepare->graph_start
636:09-12 04:02:49.445   862  4476 D AGM: graph: graph_open: 570
graph_handle 0x47534c
641:09-12 04:02:49.449   862  1489 D AGM: graph: graph_prepare: 650 entry
graph_handle 0x47534c
642:09-12 04:02:49.449   862  1489 D AGM: graph_module:
configure_hw_ep_media_config: 611 entry mod tag c0000005 miid 43d1 mid
700102a
643:09-12 04:02:49.449   862  1489 E AGM: graph_module:
configure_hw_ep_media_config: 641 rate 8000 bw 16 ch 1, data_fmt 7
655:09-12 04:02:49.462   862  1489 D AGM: graph: graph_start: 737 entry
graph_handle 0x47534c

//2. rx graph : 0x147534c
662:09-12 04:02:49.486   862  1489 D AGM: graph: graph_open: 425 entry
//configure the write shared-memory EP
//tkv: c0000002 = stream_input_media_format
663:09-12 04:02:49.492   862  1489 D AGM: graph: graph_open: 492 miid 4077
mid 7001000 tag c0000002
//configure RX HW EP
//#define DEVICE_HW_ENDPOINT_RX        0xC0000004
664:09-12 04:02:49.492   862  1489 D AGM: graph: graph_open: 537 miid 4272
mid 7001029 tag c0000004
//configure the softvolume module
//tkv: c0000006 = stream_pause
665:09-12 04:02:49.492   862  1489 D AGM: graph: graph_open: 492 miid 407e
mid 7001019 tag c0000006
//configure the PCM converter module
//stream_pcm_converter
666:09-12 04:02:49.492   862  1489 D AGM: graph: graph_open: 492 miid 407a
mid 7001003 tag c0000010
//configure the decoder module
//#define TAG_STREAM_PLACEHOLDER_DECODER  0xC0000012
667:09-12 04:02:49.493   862  1489 D AGM: graph: graph_open: 492 miid 4082
mid 7001009 tag c0000012
//configure the SPR module
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
668:09-12 04:02:49.493   862  1489 D AGM: graph: graph_open: 492 miid 4164
mid 7001032 tag c0000013

//graph_open->graph_prepare->graph_start
674:09-12 04:02:49.522   862  1489 D AGM: graph: graph_open: 570
graph_handle 0x147534c
729:09-12 04:02:49.588   862  4476 D AGM: graph: graph_prepare: 650 entry
graph_handle 0x147534c
730:09-12 04:02:49.588   862  4476 D AGM: graph_module:
set_compressed_media_format: 936 FLAC payload: ch:2, sample_size:24,
SR:96000

//configure RX HW EP, tkv = 0xC0000004
731:09-12 04:02:49.589   862  4476 D AGM: graph_module:
configure_hw_ep_media_config: 611 entry mod tag c0000004 miid 4272 mid
7001029
732:09-12 04:02:49.590   862  4476 E AGM: graph_module:
configure_hw_ep_media_config: 641 rate 96000 bw 16 ch 1, data_fmt 7
734:09-12 04:02:49.591   862  4476 D AGM: graph_module:
configure_slimbus_ep: 533 entry mod tag c0000004 miid 4272 mid 7001029
735:09-12 04:02:49.592   862  4476 D AGM: graph_module:
configure_slimbus_ep: 577 slimbus intf cfg dev id 0 ch 1

738:09-12 04:02:49.595   862  4476 E AGM: graph_module:
configure_placeholder_dec: 1022 enter
//FLAC Media format
739:09-12 04:02:49.595   862  4476 D AGM: graph_module:
configure_placeholder_dec: 1046 Placeholder mod TKV key:ae000000 value:
ae000007
747:09-12 04:02:49.622   862  1487 D AGM: graph: graph_start: 737 entry
graph_handle 0x147534c
```

4.3 QXDM analysis

Set the media format parameter for RAT (string “CAPI_RAT:”).

```
[8500/   2]              MSG                      00:03:27.988
QDSP6/High               [          capi_rat.c    216]  ADSP: RAT: 0x43f6:
CAPI_RAT: Set param received id 0x80010d0
[8500/   2]              MSG                      00:03:27.988
QDSP6/High               [          capi_rat.c    337]  ADSP: RAT: 0x43f6:
CAPI_RAT: Received rat media fmt with sr 48000, num ch 2, bps 32, data fmt
1, qfac 27
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

SLIMbus end point module configuration media format (strings “CAPI_SLIMBUS” and slimbus_driver.c”).

```
[8500/   2]              MSG                      00:03:28.055
QDSP6/High               [    slimbus_driver.c    199]  ADSP: sample_rate
96000 bit_width 16 num_channels 1
[8500/   1]              MSG                      00:03:28.055
QDSP6/Medium             [      capi_slimbus.c    700]  ADSP: CAPI_SLIMBUS:
Created scratch buffer of size 192
[8500/   2]              MSG                      00:03:28.056
QDSP6/High               [    slimbus_driver.c    163]  ADSP:
shared_channel_mapping[0] 157 shared_channel_mapping[1] 0
shared_channel_mapping[2] 0 shared_channel_mapping[3] 0
```

Send Tx feedback information to the encoder(strings”0x43fb” and “cop_depacketizer_lib”).

```
[8500/   2]              MSG                      00:03:28.229
QDSP6/High               [cop_depacketizer_v1.cpp    166]  ADSP:
cop_depacketizer_lib: Extracted bit rate is 3
[8500/   1]              MSG                      00:03:28.230
QDSP6/Medium             [gen_topo_ctrl_port.c    423]  ADSP:
TU  :20019000: Module: 0x43fb received a control message on the ctrl port
id: 0xc0000001
```

Encoder media format configuration (string “CAPI V2 APTX ADAPTIVE ENC”).

```
[8500/   2]              MSG                      00:03:28.083
QDSP6/High               [capi_aptx_adaptive_encoder.c   2561] 0495:02:
ADSP: CAPI V2 APTX ADAPTIVE ENC: input media format settings: sampling rate
48000, num channels 2, bits per sample 32, interleaving format is 0
[8500/   2]              MSG                      00:03:28.083
QDSP6/High               [capi_aptx_adaptive_encoder.c   2563] 0495:02:
ADSP: CAPI V2 APTX ADAPTIVE ENC: input media format settings: Q-format is
Q31, bitstream format is 0x09001000
[8500/   2]              MSG                      00:03:28.083
QDSP6/High               [capi_aptx_adaptive_encoder.c   2792] 0495:02:
ADSP: capi_aptx_adaptive_enc module: getting CAPI_OUTPUT_MEDIA_V2 property
0x40003, data_format = CAPI_RAW_COMPRESSED
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 09:35:46 GMT nijia,heng,dongcheng.cn

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Encoder control port with Tx COP depacketizer (string "capi_aptx_adaptive_enc.*IMCL").

```
[8500/   2]              MSG                      00:03:27.975
QDSP6/High               [capi_aptx_adaptive_enc_utils.   1126] 0899:02:
ADSP: capi_aptx_adaptive_enc: IMCL OPEN: Ctrl port[0xC0000001] open
[8500/   2]              MSG                      00:03:28.090
QDSP6/High               [capi_aptx_adaptive_enc_utils.   1169] 0495:02:
ADSP: capi_aptx_adaptive_enc: IMCL PORT PEER CONNECTED
[8500/   2]              MSG                      00:03:28.118
QDSP6/High               [capi_aptx_adaptive_encoder.c   2129] 0495:02:
ADSP: capi_aptx_adaptive_enc: Received
IMCL_PARAM_ID_BT_SIDEBAND_ENCODER_FEEDBACK
[8500/   2]              MSG                      00:03:54.481
QDSP6/High               [capi_aptx_adaptive_enc_utils.   1262] 0495:02:
ADSP: capi_aptx_adaptive_enc: IMCL PORT CLOSED
```

Packetizer (string "COP Packetization")

```
[8500/   2]              MSG                      00:03:27.988
QDSP6/High               [cop_packetizer_v0.cpp    915]  ADSP: CoP
Packetization : out SR changed from 0 to 96000
[8500/   2]              MSG                      00:03:27.988
QDSP6/High               [cop_packetizer_v0.cpp    923]  ADSP: CoP
Packetization : out channel changed from 0 to 1
[8500/   2]              MSG                      00:03:28.131
QDSP6/High               [cop_packetizer_v0.cpp    588]  ADSP: CoP
Packetization : num_zero_bytes = 2010, total len 2012
```

Depacketizer (string "capi_cop_depack_v1")

```
[8500/   2]              MSG                      00:03:27.920
QDSP6/High               [capi_cop_depacketizer_v1_utils.cpp    375]  ADSP:
capi_cop_depack_v1: Received input mf with data format 5, SR 8000, bps 16,
num channels 1
```

Qualcomm Confidential – May Contain Trade Secrets 2022-11-15 09:55:18 GMT nijusheng@hongchen.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.4 Common issues

Issue	How to resolve/what to check for
Media format mismatch for RAT ▪ Input media format received by the RAT module does not match media format configuration provided to the module ▪ Rx path: Output media format raised by the RAT module does not match what media format is expected or supported by the encoder	▪ Check for the error print “CAPI_RAT: Media format validation failed” ▪ Verify media format propagation through the modules by searching for the print “module output media format” in the QXDM logs or through the AR log parser ▪ Configure the PCM CNV in the path before the module to ensure the media format matches
Underruns on RAT module leading to mute ▪ Underrun on a module/container port indicates that data is not available on that port on time ▪ Upstream is sending data slower, or upstream data is stalled	▪ Check for the print “underrun” ▪ This issue could be a result of: ▫ RAT media format mismatch ▫ For SWB use cases, the gate may not be opened and is dropping data. Check for the print “capi_gate: Warning!: Cannot process till container delay %d and deadline time %d have been received, dropping data” ▫ Upstream data is stalled for some reason

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 09:35:08 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

5 Customize the encoder

Refer to the sample code at \adsp_proc\avs\fwk\oem\encoder\ for an example of how to write a encoder CAPI module.

5.1 Add entry in the .json file

Add an entry in the .json file as follows:

```
{
   "lib_name"      : "sample_enc_module",
   "build"         : "STATIC_BUILD_STRIP",
   "lib_major_ver" : 1,
   "lib_minor_ver" : 0,
   "is_qurt_used"  : true,
   "amdb_info"    :
   {
      "preload"          : true,
      "rev_num"          : 1,
      "itype"            : "capi",
      "mtype"            : "encoder",
      "mid"            : "0x1000107b",
      "tag"              : "capi_example_enc_module",
      "module_name"      : "MODULE_ID_EXAMPLE_ENC",
      "fmt_id1"        : "MEDIA_FMT_ID_EXAMPLE"
   }
}
```

5.2 Add API header file path in the .scons file

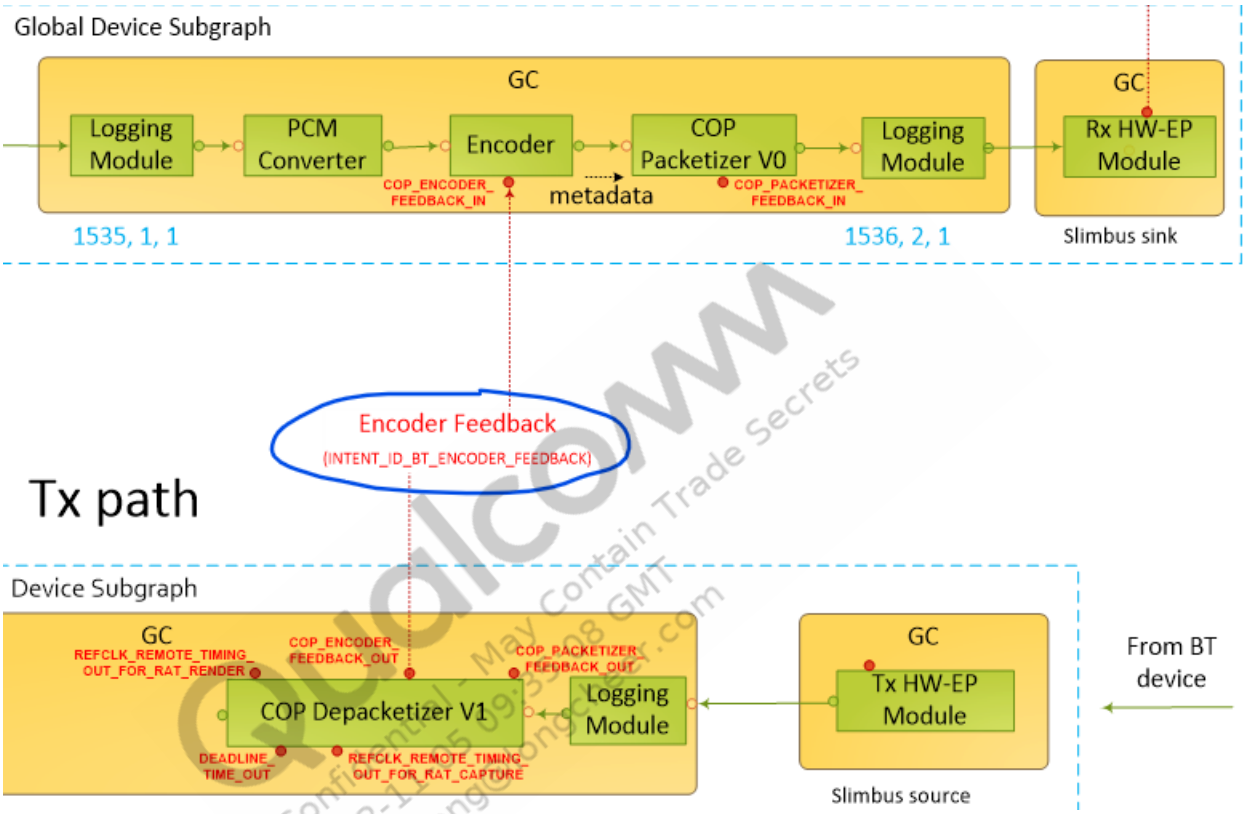
Add the following API header in sample_enc\build\example_enc_module.scons.

```
#---------------------------- h2xml generation --------------------------
env.H2xmlFileGenerate(env.subst('../api'))
#---------------------------- h2xml generation --------------------------
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 09:35:08 GMT nijiasheng@hongcheer.com

5.3 Define control port

As shown in the following diagram, a control port connection must be implemented to feed Tx rate information back to the encoder.



The module interface is defined in the CAPI API header file. Add a control port in the .h file to generate a module with a control port.

```
/* Control port for receiving br info */
+#define COP_ENCODER_FEEDBACK_IN AR_NON_GUID(0xC0000001)

/**
@h2xmlm_module              {"MODULE_ID_EXAMPLE_ENC",
MODULE_ID_EXAMPLE_ENC}
… …
    @h2xmlm_dataOutputPorts     {OUT=EXAMPLE_ENC_MODULE_DATA_OUTPUT_PORT}
+    @h2xmlm_ctrlStaticPort      {"COP_ENCODER_FEEDBACK_IN" = 0xC0000001,
                                 "Bit rate level encoder feedback intent" =
INTENT_ID_BT_ENCODER_FEEDBACK}
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

5.4 Implement control port

A control port must be implemented to communicate with the depacketizer module. For details, see *CAPI Custom Module Integration into AudioReach Signal Processing Framework User Guide* (80-VN500-17).

5.4.1 Register CAPI_INTERFACE_EXTENSIONS

In x_static_properties, implement CAPI_INTERFACE_EXTENSIONS for INTF_EXTN_IMCL and INTF_EXTN_METADATA.

```
         case CAPI_INTERFACE_EXTENSIONS:
         {
            if (payload_ptr->max_data_len >=
sizeof(capi_interface_extns_list_t))
            {
               capi_interface_extns_list_t *intf_ext_list =
(capi_interface_extns_list_t *)payload_ptr->data_ptr;
               if (payload_ptr->max_data_len <
(sizeof(capi_interface_extns_list_t) +
                                                (intf_ext_list-
>num_extensions * sizeof(capi_interface_extn_desc_t))))
               {
                  AR_MSG(DBG_ERROR_PRIO,
                         "capi_example_enc: CAPI_INTERFACE_EXTENSIONS
invalid param size %lu",
                         payload_ptr->max_data_len);
                  CAPI_SET_ERROR(capi_result, CAPI_ENEEDMORE);
               }
               else
               {
                  capi_interface_extn_desc_t *curr_intf_extn_desc_ptr =
                     (capi_interface_extn_desc_t *)(payload_ptr->data_ptr +
sizeof(capi_interface_extns_list_t));

                  for (uint32_t i = 0; i < intf_ext_list->num_extensions;
i++)
                  {
                     switch (curr_intf_extn_desc_ptr->id)
                     {
                        case INTF_EXTN_IMCL:
                        {
                           curr_intf_extn_desc_ptr->is_supported = TRUE;
                           break;
                        }
                        case INTF_EXTN_METADATA:
                        {
                           curr_intf_extn_desc_ptr->is_supported = TRUE;
```

Qualcomm Confidential – May Contain Trade Secrets 2025-11-05 08:15:08 GMT nijiashang@lomscreer.com

```
                            break;
                         }
                         default:
                         {
                            curr_intf_extn_desc_ptr->is_supported = FALSE;
                            break;
                         }
                      }
                      AR_MSG(DBG_HIGH_PRIO,
                             "capi_example_enc: CAPI_INTERFACE_EXTENSIONS
intf_ext = 0x%lx, is_supported = %d",
                             curr_intf_extn_desc_ptr->id,
                             (int)curr_intf_extn_desc_ptr->is_supported);
                      curr_intf_extn_desc_ptr++;
                   }
                }
             }
             else
             {
                AR_MSG(DBG_ERROR_PRIO,
                       "capi_example_enc: CAPI_INTERFACE_EXTENSIONS Bad
param size %lu",
                       payload_ptr->max_data_len);
                CAPI_SET_ERROR(capi_result, CAPI_ENEEDMORE);
             }
             break;
          }
```

5.4.2 Port configuration

In x_set_param, open a port (INTF_EXTN_IMCL_PORT_OPEN), connect the port (INTF_EXTN_IMCL_PORT_PEER_CONNECTED), disconnect the port (INTF_EXTN_IMCL_PORT_PEER_DISCONNECTED), and close the port (INTF_EXTN_IMCL_PORT_CLOSE).

```
      case INTF_EXTN_PARAM_ID_IMCL_PORT_OPERATION:
      {
         /** Set the control port operation */
         if (CAPI_EOK != (result =
capi_example_enc_imcl_port_operation(me_ptr, port_info_ptr, params_ptr)))
         {
            AR_MSG(DBG_ERROR_PRIO, "capi_example_enc: Failed to set control
port operation with result %d", result);
         }
         break;
      }

//port operation encapsulation
```

Qualcomm Confidential – May Contain Trade Secrets 2022-11-05 09:55:08 GMT nijiasheng@longcheer.com

```
capi_err_t capi_example_enc_imcl_port_operation(capi_example_enc_module_t *
me_ptr,
                                                const capi_port_info_t
*port_info_ptr,
                                                capi_buf_t *
params_ptr)
{
   capi_err_t                  capi_result   = CAPI_EOK;
   intf_extn_imcl_port_open_t *port_open_ptr = NULL;

   if (params_ptr->actual_data_len <
sizeof(intf_extn_param_id_imcl_port_operation_t))
   {
      AR_MSG(DBG_ERROR_PRIO,
             "capi_example_enc: Invalid payload size[%lu] for ctrl port
operation",
             params_ptr->actual_data_len);
      return CAPI_ENEEDMORE;
   }

   intf_extn_param_id_imcl_port_operation_t *port_op_ptr =
      (intf_extn_param_id_imcl_port_operation_t *)(params_ptr->data_ptr);

   if (!port_op_ptr->op_payload.data_ptr)
   {
      AR_MSG(DBG_ERROR_PRIO,
             "capi_example_enc: NULL payload for ctrl port open/close
opcode [0x%lX]",
             port_op_ptr->opcode);
      return CAPI_EBADPARAM;
   }

   switch (port_op_ptr->opcode)
   {
      case INTF_EXTN_IMCL_PORT_OPEN:
      {
         port_open_ptr = (intf_extn_imcl_port_open_t *)port_op_ptr-
>op_payload.data_ptr;

         /** Get the number of control ports being opened */
         uint32_t num_ports = port_open_ptr->num_ports;

         if (num_ports > EXAMPLE_ENC_MAX_CONTROL_PORTS)
         {
            AR_MSG(DBG_ERROR_PRIO,
                   "capi_example_enc: IMCL CONNECT: example module only
supports one control port. Trying to "
                   "connect %lu",
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 09:35:08 GMT mjaiwengolongcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
                num_ports);
            return CAPI_EBADPARAM;
         }

         /** Size Validation*/
         uint32_t valid_size =
            sizeof(intf_extn_imcl_port_open_t) + (num_ports *
sizeof(intf_extn_imcl_id_intent_map_t));

         if (port_op_ptr->op_payload.actual_data_len < valid_size)
         {
            AR_MSG(DBG_ERROR_PRIO,
                   "capi_example_enc: Invalid payload size[%lu] for ctrl
port OPEN",
                   params_ptr->actual_data_len);
            return CAPI_ENEEDMORE;
         }

         /** Iterate over the list of all the control ports being opened */
         for (uint32_t iter = 0; iter < num_ports; iter++)
         {
            /** Validate the number of intents per control port   */
            if (port_open_ptr->intent_map[iter].num_intents >
EXAMPLE_ENC_MAX_INTENTS_PER_CTRL_PORT)
            {
               AR_MSG(DBG_ERROR_PRIO,
                      "capi_example_enc: IMCL OPEN: Expected num
intent[%lu], Configured num intent[%lu]",
                      EXAMPLE_ENC_MAX_INTENTS_PER_CTRL_PORT,
                      port_open_ptr->intent_map[iter].num_intents);
               return CAPI_EBADPARAM;
            }

            /** Size Validation*/
            valid_size += (port_open_ptr->intent_map[iter].num_intents *
sizeof(uint32_t));

            /** Validate the actual payload length */
            if (port_op_ptr->op_payload.actual_data_len < valid_size)
            {
               AR_MSG(DBG_ERROR_PRIO,
                      "capi_example_enc: Invalid payload size[%lu] for ctrl
port OPEN",
                      params_ptr->actual_data_len);
               return CAPI_ENEEDMORE;
            }
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
            uint32_t ctrl_port_id = port_open_ptr-
>intent_map[iter].port_id;

            if (COP_ENCODER_FEEDBACK_IN != ctrl_port_id)
            {
               AR_MSG(DBG_ERROR_PRIO,
                      "capi_example_enc: IMCL OPEN: Error, unsupported
control port id %d  ",
                      ctrl_port_id);
               return CAPI_EBADPARAM;
            }

            if (CTRL_PORT_OPEN == me_ptr->in_ctrl_port_info.state)
            {
               AR_MSG(DBG_ERROR_PRIO, "capi_example_enc: OPEN: Error, Ctrl
port[0x%lX] already opened", ctrl_port_id);
               return CAPI_EFAILED;
            }

            /** Intent ID check */
            for (uint32_t idx = 0; idx < port_open_ptr-
>intent_map[iter].num_intents; idx++)
            {
               if (INTENT_ID_BT_ENCODER_FEEDBACK != port_open_ptr-
>intent_map[iter].intent_arr[idx])
               {
                  AR_MSG(DBG_ERROR_PRIO,
                         "capi_example_enc: Invalid intent id %x",
                         port_open_ptr->intent_map[iter].intent_arr[idx]);
                  capi_result |= CAPI_EBADPARAM;
                  continue;
               }
            }

            /* Save the values */
            me_ptr->in_ctrl_port_info.port_id = ctrl_port_id;
            me_ptr->in_ctrl_port_info.state   = CTRL_PORT_OPEN;

            AR_MSG(DBG_HIGH_PRIO, "capi_example_enc: IMCL OPEN: Ctrl
port[0x%lX] open", ctrl_port_id);

         } /** End of for loop (num ports) */
         break;
      } /** End of case INTF_EXTN_IMCL_PORT_OPEN */
      case INTF_EXTN_IMCL_PORT_PEER_CONNECTED:
      {
         if (port_op_ptr->op_payload.data_ptr)
         {
```

Qualcomm Confidential – May Contain Trade Secrets 2022-11-05 03:33:08 GMT miaoyeqi@longcheer.com

```
            intf_extn_imcl_port_start_t *port_start_ptr =
               (intf_extn_imcl_port_start_t *)port_op_ptr-
>op_payload.data_ptr;
            /*Size Validation*/
            uint32_t num_ports = port_start_ptr->num_ports;

            if (num_ports > EXAMPLE_ENC_MAX_CONTROL_PORTS)
            {
               AR_MSG(DBG_ERROR_PRIO,
                      "capi_example_enc: IMCL CONNECT: example module only
supports one control port. Trying to "
                      "connect %lu",
                      num_ports);
               return CAPI_EBADPARAM;
            }

            uint32_t valid_size = sizeof(intf_extn_imcl_port_start_t) +
(num_ports * sizeof(uint32_t));
            if (port_op_ptr->op_payload.actual_data_len < valid_size)
            {
               AR_MSG(DBG_ERROR_PRIO,
                      "capi_example_enc: IMCL CONNECT: Invalid payload size
for ctrl port peer connect %d",
                      params_ptr->actual_data_len);
               return CAPI_ENEEDMORE;
            }

            // for each port id in the list that follows
            for (uint32_t iter = 0; iter < num_ports; iter++)
            {
               if (COP_ENCODER_FEEDBACK_IN != port_start_ptr-
>port_id_arr[iter])
               {
                  AR_MSG(DBG_ERROR_PRIO,
                         "capi_example_enc: IMCL CONNECT: Ctrl port id %lx
mapping not found.",
                         port_start_ptr->port_id_arr[iter]);
                  return CAPI_EBADPARAM;
               }
               me_ptr->in_ctrl_port_info.state = CTRL_PORT_PEER_CONNECTED;
            }
         }
         else
         {
            AR_MSG(DBG_ERROR_PRIO, "capi_example_enc: IMCL CONNECT: Ctrl
port peer connected expects a payload. Failing.");
            return CAPI_EBADPARAM;
         }
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 03:35:08 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
         break;
      }
      case INTF_EXTN_IMCL_PORT_PEER_DISCONNECTED:
      {
         if (port_op_ptr->op_payload.data_ptr)
         {
            intf_extn_imcl_port_stop_t *port_start_ptr =
(intf_extn_imcl_port_stop_t *)port_op_ptr->op_payload.data_ptr;
            /*Size Validation*/
            uint32_t num_ports = port_start_ptr->num_ports;

            if (num_ports > EXAMPLE_ENC_MAX_CONTROL_PORTS)
            {
               AR_MSG(DBG_ERROR_PRIO,
                      "capi_example_enc: IMCL DISCONNECT: example module
only supports %d control port. Trying to "
                      "disconnect %lu",
                      EXAMPLE_ENC_MAX_CONTROL_PORTS,
                      num_ports);
               return CAPI_EBADPARAM;
            }

            uint32_t valid_size = sizeof(intf_extn_imcl_port_stop_t) +
(num_ports * sizeof(uint32_t));
            if (port_op_ptr->op_payload.actual_data_len < valid_size)
            {
               AR_MSG(DBG_ERROR_PRIO,
                      "capi_example_enc: IMCL DISCONNECT: Invalid payload
size for ctrl port peer disconnect %d",
                      params_ptr->actual_data_len);
               return CAPI_ENEEDMORE;
            }

            // for each port id in the list that follows
            for (uint32_t iter = 0; iter < num_ports; iter++)
            {
               if (COP_ENCODER_FEEDBACK_IN != port_start_ptr-
>port_id_arr[iter])
               {
                  AR_MSG(DBG_ERROR_PRIO,
                         "capi_example_enc: IMCL DISCONNECT: Ctrl port %lu
mapping not found.",
                         port_start_ptr->port_id_arr[iter]);
                  return CAPI_EBADPARAM;
               }
               me_ptr->in_ctrl_port_info.state =
CTRL_PORT_PEER_DISCONNECTED;
            }
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
         }
         else
         {
            AR_MSG(DBG_ERROR_PRIO, "capi_example_enc: IMCL DISCONNECT:
expects a payload.Failing.");
            return CAPI_EBADPARAM;
         }
         break;
      }
      case INTF_EXTN_IMCL_PORT_CLOSE:
      {
         intf_extn_imcl_port_close_t *port_close_ptr =
(intf_extn_imcl_port_close_t *)port_op_ptr->op_payload.data_ptr;

         /** Size Validation*/
         uint32_t num_ports = port_close_ptr->num_ports;
         if (!num_ports)
         {
            AR_MSG(DBG_HIGH_PRIO, "capi_example_enc: IMCL CLOSE: Warning!
Num ctrl ports to be closed is zero");
            return CAPI_EOK;
         }

         if (num_ports > EXAMPLE_ENC_MAX_CONTROL_PORTS)
         {
            AR_MSG(DBG_ERROR_PRIO,
                   "capi_example_enc: IMCL CLOSE: example module only
supports one control port. Trying to "
                   "close %lu",
                   num_ports);
            return CAPI_EBADPARAM;
         }

         uint32_t valid_size = sizeof(intf_extn_imcl_port_close_t) +
(num_ports * sizeof(uint32_t));

         if (port_op_ptr->op_payload.actual_data_len < valid_size)
         {
            AR_MSG(DBG_ERROR_PRIO, "capi_example_enc: IMCL CLOSE: Invalid
payload size %d", params_ptr->actual_data_len);
            return CAPI_ENEEDMORE;
         }

         /** Iterate over the list of port ID's  */
         for (uint32_t iter = 0; iter < num_ports; iter++)
         {
            /** Get the control port ID to be closed */
            uint32_t ctrl_port_id = port_close_ptr->port_id_arr[iter];
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
            if (COP_ENCODER_FEEDBACK_IN != ctrl_port_id)
            {
               AR_MSG(DBG_ERROR_PRIO,
                      "capi_example_enc: IMCL CLOSE: Error, unsupported
control port id %lx ",
                      ctrl_port_id);
               return CAPI_EBADPARAM;
            }

            /* Save the values */
            me_ptr->in_ctrl_port_info.state = CTRL_PORT_CLOSE;
         } /** End of for() */
         break;
      } /** End of case INTF_EXTN_IMCL_PORT_CLOSE */
      default:
      {
         AR_MSG(DBG_ERROR_PRIO,
                "capi_example_enc: Set, unsupported IMC port operation
opcode ID[0x%lX]",
                port_op_ptr->opcode);
         capi_result = CAPI_EUNSUPPORTED;
         break;
      }
   } /** End of switch (port_op)) */
   return capi_result;
}
```

5.4.3 Receiving data from sender

In the set param operation, data is received from the sender by via the INTF_EXTN_PARAM_ID_IMCL_INCOMING_DATA parameter.

```
      case INTF_EXTN_PARAM_ID_IMCL_INCOMING_DATA:
      {
         if (params_ptr->actual_data_len <
sizeof(intf_extn_param_id_imcl_incoming_data_t))
         {
            AR_MSG(DBG_ERROR_PRIO,
                   "capi_example_enc: Invalid payload size for param %x,
size %d",
                   param_id,
                   params_ptr->actual_data_len);
            return CAPI_ENEEDMORE;
         }

         intf_extn_param_id_imcl_incoming_data_t *payload_ptr =
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 09:35:08 GMT nijusheng@longcheer.com

```
            (intf_extn_param_id_imcl_incoming_data_t *)params_ptr-
>data_ptr;

         imc_param_header_t *imc_cfg_hdr_ptr = (imc_param_header_t
*)(payload_ptr + 1);

         switch (imc_cfg_hdr_ptr->opcode)
         {
            case IMCL_PARAM_ID_BT_BIT_RATE_LEVEL_ENCODER_FEEDBACK:
            {
               AR_MSG(DBG_HIGH_PRIO, "capi_example_enc: Received
IMCL_PARAM_ID_BT_BIT_RATE_LEVEL_ENCODER_FEEDBACK");
               if (imc_cfg_hdr_ptr->actual_data_len <
sizeof(imcl_param_id_bt_bitrate_encoder_feedback_t))
               {
                  AR_MSG(DBG_ERROR_PRIO,
                         "capi_example_enc: actual_data_len %lu size of
bitrate encoder feedback payload %u ",
                         params_ptr->actual_data_len,

sizeof(imcl_param_id_bt_bitrate_encoder_feedback_t));
                  return CAPI_ENEEDMORE;
               }

               imcl_param_id_bt_bitrate_encoder_feedback_t *br_level =
                  (imcl_param_id_bt_bitrate_encoder_feedback_t
*)(imc_cfg_hdr_ptr + 1);

               me_ptr->cur_br_level = br_level->bit_rate_level; // caching
new bit rate level

               AR_MSG(DBG_HIGH_PRIO,
                      "capi_example_enc: bit rate level received %ld set
example bitrate to %u ",
                      br_level->bit_rate_level,
                      me_ptr->cur_br_level);
                //TODO, Reinitialize encoder, update KPPS and BANDWIDTH
according to the new bit rate
               result = CAPI_EOK;

               break;
            }
            default:
            {
               AR_MSG(DBG_ERROR_PRIO,
                      "capi_example_enc: IMCL port incoming data,
Unsupported Param id :0x%x \n",
                      imc_cfg_hdr_ptr->opcode);
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-03 02:35:08 GMT nijiasheng@lovelycheer.com

```
                break;
            }
         }
         break;
      }
```

5.4.4 Sending data to the receiver

The COP Depacketizer sends the bit rate information via the control port to the Bluetooth encoder module.

```
capi_err_t capi_cop_depack_send_bitrate_info(capi_cop_depack_v1_t *me_ptr,
uint32_t bitrate)
{

   if (CTRL_PORT_PEER_CONNECTED != me_ptr-
>out_ctrl_port_arr[COP_ENC_PORT_IDX].state)
   {
      AR_MSG(DBG_ERROR_PRIO, "capi_cop_depack_v1: Control port not in
connected state, cannot send bitrate level");
      return CAPI_EFAILED;
   }

   imcl_param_id_bt_bitrate_encoder_feedback_t bitrate_info = { 0 };

   bitrate_info.bit_rate_level = bitrate;
   imc_param_header_t header;
   header.opcode          =
IMCL_PARAM_ID_BT_BIT_RATE_LEVEL_ENCODER_FEEDBACK;
   header.actual_data_len =
sizeof(imcl_param_id_bt_bitrate_encoder_feedback_t);

   /*Get a recurring buf from the fwk*/
   capi_buf_t buffer;

   capi_cmn_imcl_get_recurring_buf(&me_ptr->cb_info,
COP_ENCODER_FEEDBACK_OUT, &buffer);
   if (NULL == buffer.data_ptr)
   {
      AR_MSG(DBG_ERROR_PRIO, "capi_cop_depack_v1: received NULL payload ptr
while sending bitrate feedback to Enc");
      return CAPI_EFAILED;
   }

   /*Copy the imcl_header*/
   uint32_t bytes_copied = memscpy(buffer.data_ptr, buffer.max_data_len,
&header, sizeof(header));
```

Qualcomm Confidential – May Contain Trade Secrets 2022-11-15 09:25:08 GMT jinsheng@zangcomer.com

```
   /*Followed by the bitrate payload*/
   bytes_copied += memscpy(buffer.data_ptr + bytes_copied,
                           buffer.max_data_len - bytes_copied,
                           ((int8_t *)&bitrate_info),
                           header.actual_data_len);

   buffer.actual_data_len = bytes_copied;

   /*To send data over to the IMCL peer*/
   imcl_outgoing_data_flag_t flags;
   flags.should_send = TRUE;
   flags.is_trigger  = FALSE;

#ifdef COP_DEBUG
   AR_MSG(DBG_HIGH_PRIO, "capi_cop_depack_v1: IMCL sending bitrate
level %d", bitrate);
#endif

   return capi_cmn_imcl_send_to_peer(&me_ptr->cb_info, &buffer,
COP_ENCODER_FEEDBACK_OUT, flags);
}
```

5.5 Metadata

1. Define the interface extension used to send metadata information to the COP Packetizer:

```
#define INTF_EXTN_METADATA 0x0A00101F
```

2. Set the following parameter to register for the metadata handler:

```
#define INTF_EXTN_PARAM_ID_METADATA_HANDLER 0x0A001020
```

3. Propagate metadata by calling the metadata_handler.metadata_propagate callback.
4. Implement sending of the following metadata IDs:
   - To inform the packetizer of the media format (sample rate, bits per sample, number of channels) at which data was encoded.

     ```
     #define MD_ID_COP_PCM_MEDIA_FORMAT 0x0A001038
     ```

     If this metadata is not implemented, the packetizer will not be able to process data.

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 09:35:08 GMT nijiasheng@longsheer.com

- To send information about number of PCM samples (per channel) required to produce one encoded frame:

  ```
  #define MD_ID_COP_PCM_FRAME_LENGTH 0x0A001037
  ```

  If this metadata is not implemented, the packetizer will not be able to process data.

- To have the encoder configure the packetizer to enable/disable zero padding:

  ```
  #define MD_ID_COP_CONFIG_ZERO_PADDING 0x0A001039
  ```

  If this metadata is not implemented, the packetizer assumes zero padding is enabled.

5. Send metadata by calling the metadata_handler.metadata_create callback.

5.6 AP side customization

1. Add a new GKV value for Bluetooth format in kvh2xml.h.

```
enum Key_BtFormat {
    GENERIC      = 0xB5000001, /**< @h2xmle_name {GENERIC}*/
    LDAC         = 0xB5000002, /**< @h2xmle_name {LDAC}*/
    APTX_ADAPTIVE = 0xB5000003, /**< @h2xmle_name {APTX_ADAPTIVE}*/
    SWB          = 0xB5000004, /**< @h2xmle_name {SWB}*/
};
```

2. Add customization code in Bluetooth.cpp by following the reference format (LDAC or APTX).
3. Generate the kv2hxml.xml.

```
h2xml -conf "path of <h2xm_config_casa.xml>" -o .\ -t Key kvh2xml.h
```

Qualcomm Confidential - May Contain Trade Secrets 2022-11-09 17:35:08 GMT yijiashengcloudcacher.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

5.7 ACDB design

1. Import the new KV from new kv2hxml.xml for the customization format.
2. Import the newly added module.
3. Design the subgraph and use case by referencing the LDAC or APTX path.
4. Flash a new ADSP image with the lib integrated.
5. Push the ACDB files with the customization and PAL code changes.
6. Test if it the module can be run.
7. Fix any AP code errors using logcat logs.
8. Fix any ADSP code errors using QXDM logs.

Qualcomm Confidential - May Contain Trade Secrets 2022-11-05 09:35:08 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

A References

A.1 Related documents

Title	Number
Qualcomm Technologies, Inc.	
CAPI Custom Module Integration into AudioReach Signal Processing Framework User Guide	80-VN500-17

A.2 Acronyms and terms

Acronym or term	Definition
AP	Access Point
RAT	Rate Adapted Timer
COP	Compressed over PCM

Qualcomm
Confidential – May Contain Trade Secrets
2022-11-05 09:35:08 GMT
nijiasheng@longcheer.com